

Practical Secret-Key Recovery against Bittersweet Level I

Accepted Carry Comparisons and Weighted Lattice Recovery

ePrint Signature Audit
AI-assisted cryptanalysis report

8 October 2026

Abstract

Bittersweet is an LWR-based MPC-in-the-head signature whose optimized responses disclose an opened key-share aggregate and a one-bit reconstruction error. At the published Level-I $d = 32$ parameters, those values reveal an oriented strict comparison with a fixed 90-bit buffer of every secret LWR product. We collect 16 accepted signatures, giving 416 comparisons for each of 747 public rows, and intersect them into modular product intervals. A weighted modular closest-vector instance on the 160 narrowest intervals is solved with plain LLL and Babai nearest plane.

In a precommitted held-out execution and a separate fresh-context reproduction, each public-only candidate satisfied all 747 intervals and rounded public outputs before private truth was opened. Hash-pinned comparators reported an exact match in all eleven 96-bit key words. Each recovered candidate produced a signature on a distinct seventeenth message. The ordinary strict verifier accepted all 26 repetitions and all 747 rows per repetition, while ten final mutations rejected. The reproduction used exactly one new key and one attack trial. Its public pipeline took 313.95 seconds and at most 236,760 KiB RSS on one worker.

The result concerns ePrint 2026/397 version 1’s Level-I $d = 32$ row under a frozen canonical realization of the paper’s arithmetic and protocol structure. The paper publishes no author implementation or normative byte format. Fresh-context reproduction and the dated final history search have passed. Independent human verification is pending.

Status: AI_REPRODUCED_HUMAN_PENDING. Run-003 and independent run-005 pass the practical full-parameter recovery and fresh-signing checks. The final dated attack-history recheck also passes. Independent human verification is pending.

1 Introduction

Bittersweet [1] builds an MPC-in-the-head signature from a power-of-two Learning With Rounding (LWR) function. Its public key contains a three-bit rounding of a matrix–secret product. To reconcile additive key sharing with rounding, each optimized response includes a one-bit correction for every public row. Signing aborts when a 90-bit slice of the opened-share product equals the corresponding slice of the full secret product.

The target paper explicitly discusses carry leakage and observes that an accepted transcript excludes buffer values. It heuristically posits that the resulting leakage remains harmless when the signing-query count is very small relative to 2^{90} . We show that far fewer queries suffice at the claimed Level-I parameters. The correction bit gives the direction of every excluded threshold. Repeating the comparison 416 times per row yields intervals narrow enough to recover the complete key with conventional lattice tools.

Our concrete contributions are:

- an exact derivation from each accepted optimized response to an oriented strict comparison with the fixed secret-product buffer;
- an interval accumulator using all 16 signatures and 26 repetitions, for 310,752 public comparisons;
- a practical weighted modular CVP instantiation using 160 rows, plain LLL [2], and Babai nearest plane [3];
- a precommitted held-out execution in which the candidate is frozen and independently replayed before truth access;

- a fresh-context one-key, one-trial reproduction with independent public-only result replay; and
- a complete fresh-message signature accepted by the ordinary strict verifier, together with mutation controls.

No originality is claimed for LLL, Babai, generic modular-CVP embeddings, power-of-two LWR cryptanalysis, or the general existence of carry leakage. We describe the modular-interval and accepted-transcript steps as conventional or target-disclosed components and make no separate novelty claim for them.

Attack-history status. No prior public attack against Bittersweet Signatures was located in the searches performed as of 2026-10-07. The final post-reproduction search identified ePrint 2026/250 [4] as generic prior power-of-two-LWR technique, not an attack on exact Bittersweet. This dated finding is not an absolute first-attack claim.

2 Target and paper-level model

The exact target is IACR ePrint 2026/397 version 1, whose pinned PDF has SHA-256

eefe27b903cb61fa50fecc33eef3fa99b121bece1c9831a4b3cab7bdebba5dbb.

We execute Table 2’s Level-I $d = 32$ row:

$$(\lambda, q, p', \alpha, p, n, m, d, t) = (128, 96, 3, 90, 93, 11, 747, 32, 26),$$

where $p = p' + \alpha$. The secret is $k \in \mathbb{Z}_{296}^{11}$. A public input is expanded to $X \in \mathbb{Z}_{296}^{747 \times 11}$, and the public rounded output is

$$y = \lfloor Xk \rfloor_{23}.$$

In each repetition the signer shares k among 32 parties, commits to the shares, computes local LWR outputs, and opens all but one share selected by the Fiat–Shamir challenge. The optimized signature transmits GGM material or a correction key for the opened shares, the hidden commitment, and a one-bit reconstruction-error vector. The verifier reconstructs the opened aggregate, uses the bit to rebuild the hidden output, rehashes the complete first message, and applies Figure 3’s modular predicate.

The ePrint supplies no public implementation, normative wire encoding, domain-separation byte strings, XOF-to-matrix convention, or test vectors. Our frozen model uses injective encodings and conventional SHAKE/SHA3 components, implements both optimized GGM opening cases, checks packed-bit padding, and preserves the paper’s exact arithmetic and Fiat–Shamir bindings. Our claim is about that mathematical design. We do not claim byte compatibility with an unavailable implementation.

3 Accepted transcripts reveal strict comparisons

Fix a public row X_r and write its secret product as

$$\begin{aligned} u_r &= \langle X_r, k \rangle \bmod 2^{96}, \\ u_r &= y_r 2^{93} + R_r, \\ R_r &= 8B_r + \ell_r, \end{aligned}$$

where $0 \leq B_r < 2^{90}$ and $0 \leq \ell_r < 8$. The three-bit y_r is public. The 90-bit buffer B_r is fixed by the public row and signing key, and is therefore reused across all messages, salts, signatures, and repetitions.

For one response, let K_{open} be the verifier-reconstructed sum of the 31 opened key shares. Define

$$S_r = \langle X_r, K_{\text{open}} \rangle \bmod 2^{96}, \quad A_r = (S_r \gg 3) \bmod 2^{90}.$$

The signing abort rejects equality $A_r = B_r$. The one-bit reconstruction error e_r , interpreted through Figure 3’s modular membership rule, supplies the orientation:

$$\begin{aligned} e_r = 0 &\implies A_r < B_r \implies B_r \geq A_r + 1, \\ e_r = 1 &\implies A_r > B_r \implies B_r \leq A_r - 1. \end{aligned}$$

Observation 1. *Every accepted repetition gives one exact strict comparison per public row with a row-specific fixed buffer. It does not reveal 90 independent bits, and the 747 comparisons in one repetition remain coupled through the same opened key aggregate.*

The derivation uses the stricter formal Figure 3 rule. Section 7.1’s prose inequality is ambiguous if read as an ordinary signed comparison; our attack does not rely on that broader reading.

4 Accumulating modular product intervals

Initialize $L_r = 0$ and $U_r = 2^{90} - 1$. For every signature and repetition, update

$$\begin{aligned} e_r = 0 : \quad & L_r \leftarrow \max(L_r, A_r + 1), \\ e_r = 1 : \quad & U_r \leftarrow \min(U_r, A_r - 1). \end{aligned}$$

Sixteen signatures give $16 \cdot 26 = 416$ comparisons per row and $416 \cdot 747 = 310,752$ comparisons overall. The final buffer bracket implies

$$y_r 2^{93} + 8L_r \leq \langle X_r, k \rangle \bmod 2^{96} \leq y_r 2^{93} + 8(U_r + 1) - 1. \quad (1)$$

The last seven values in the upper endpoint account for the unknown low three bits.

Run-003 and run-005 each began from 16 complete serialized optimized signatures on distinct messages. The ordinary strict verifier accepted all of them. Independent replay reconstructed every opened aggregate and packed error vector and reproduced the 747 intervals. Run-003 recorded 171/171 applicable query-signature controls; run-005 separately recorded 169/169. No secret label or private truth is needed to compute Equation (1).

5 Weighted lattice recovery

For each row there exists an integer lift z_r such that

$$\langle X_r, k \rangle - 2^{96} z_r \in [\text{lower}_r, \text{upper}_r]. \quad (2)$$

Let c_r and w_r be the midpoint and width of this interval. The frozen procedure selects the 160 narrowest rows and assigns

$$\omega_r = \max \left(1, \text{round} \left(\frac{\max_s w_s}{w_r} \right) \right).$$

It embeds the weighted modular equations and eleven systematic key coordinates as a closest-vector problem centered on the interval midpoints. Plain LLL with $\delta = 0.99$ and $\eta = 0.501$ reduces the basis, after which Babai nearest plane produces a candidate. Every candidate is checked against all 747 intervals and all 747 public rounded outputs; truth is not part of the stop rule.

The final schedule was fixed before held-out entropy:

Accepted signatures	16
Selected rows	160 narrowest
Block schedule	[0] (plain LLL, then Babai)
Configured loops	2
fpylll seed	20260412
Stop rule	first candidate satisfying every public constraint

Run-003 made one block-zero attempt. The complete solver took 45.674 seconds with 235,768 KiB peak RSS. Before truth access, the candidate fit 160/160 selected intervals, 747/747 total intervals, and 747/747 public rounded outputs.

Run-005 independently executed the same family-B schedule exactly once, with one public-pipeline invocation, one block-zero attempt, and no tuning or retry. Its candidate also fit 160/160 selected intervals, 747/747 total intervals, and 747/747 rounded outputs. A separate public-only reviewer rebuilt all 310,752 comparisons and replayed the solver and candidate constraints.

Table 1: Run-003 resource measurements; all stages used one worker.

Stage	Wall time	Peak RSS
Public verification, aggregation, recovery, freeze	320.47 s	236,792 KiB
Truth comparison	0.20 s	26,800 KiB
Recovered-key fresh signing	2.25 s	28,056 KiB
Fresh verification and controls	15.04 s	29,696 KiB

6 Final executions and fresh signing

The target, sources, runtime, query rule, solver schedule, resource limits, public/private boundary, truth comparator, message rule, signer, and verifier were frozen before the run-003 held-out entropy existed. A new key and 16 complete signatures were generated. The public pipeline verified and aggregated them, ran the one configured recovery attempt, and serialized a public candidate freeze. Independent public-only and adversarial reviewers replayed that chain before authorizing the truth comparison.

The hash-pinned comparator reported:

- exact equality in all eleven 96-bit words;
- identical candidate and truth key digests;
- identical 747-product digests;
- 747/747 candidate and truth interval checks;
- 747/747 public-output checks; and
- exact public-key reconstruction.

It emitted no raw key words. The recovered/truth key digest is

8b7136532fc8be787564db67d2bbdf8e027dba822cfc0412377d96f92df426be.

The fresh-message derivation rule was also frozen before held-out entropy. It produced a 140-byte seventeenth message, SHA-256

22fd90f30a746ab8dbffa4041b2ceba0e2c2988241d624fcc664bbd53ae09f2e,

distinct from every oracle query. The recovered public candidate produced a complete 26-response signature in one attempt with zero aborts. The ordinary strict verifier accepted all repetitions and all 747 rows per repetition.

Ten applicable mutations rejected: challenge index, message, public matrix, public rounded output, correction key, error bit, GGM path, hidden commitment, salt, and nonzero packed-error padding. A strengthened changed-message test updated its noncryptographic envelope metadata and still rejected at the recomputed Fiat–Shamir first-message hash. An independent replay also bound the mathematical signature body directly to the original public-key artifact.

The sealed public resource summary supports the 320.47-second public pipeline and the 45.674-second complete solver within it. It does not bind a separate key-and-signature-generation duration or a complete run-003 stage sum. Review and documentation time is excluded.

6.1 Independent run-005

The first reproduction campaign, run-004, failed before entropy: an inherited standard-input version probe entered an interactive Singular prompt. It ended after 32.19 seconds at 228,420 KiB RSS with zero keys, signing queries, verifier calls, solver trials, truth comparisons, and fresh signatures. The fix closed standard input, captured output, and imposed a finite timeout. An initial run-005 source review then VETOed a mismatch between the declared first-nonempty-line parser and the implementation. Rev2 fixed the parser and passed before entropy.

Run-005 generated one new OS-random key and 16 complete signatures on distinct messages with zero oracle aborts. Its single attack trial produced 747/747 interval and rounded-output checks, followed by 11/11 exact words and 747/747 truth intervals through the sealed comparator. It signed one distinct 140-byte message, whose SHA-256 is

00e047a068069375376ff8b812ea98098f89ec9a8f0ee7937e818b4ba14ab9d4.

Table 2: Run-005 resource measurements; every stage used one worker.

Stage	Wall time	Peak RSS
Main precommit	2.60 s	230,304 KiB
Seed/commitment generation	2.82 s	230,496 KiB
Fresh key and 16 signatures	42.80 s	234,948 KiB
Public verification, aggregation, recovery, freeze	313.95 s	236,760 KiB
Truth comparison	0.20 s	26,840 KiB
Recovered-key fresh signing	2.25 s	28,024 KiB
Fresh verification and controls	15.20 s	29,676 KiB

The signature was produced in one attempt with zero aborts. The strict verifier accepted 26/26 repetitions and all 747 rows, and ten final controls rejected. A separate public-only result reviewer replayed the comparison, solver, candidate, and fresh-verifier constraints without opening private state.

The seven-stage sum is 379.82 seconds under the 4 GiB cap.

7 Trial accounting

The empirical record contains heterogeneous, adaptive, and repeated-key procedures and cannot be reduced to one binomial success estimate.

Original projected development.

One tuned training key has two documented successful configurations, S4/r120 and an independently frozen S8 configuration. Earlier tuning attempts are not exhaustively enumerated; this is adaptive development, not campaign evidence.

Fresh adaptive S8 development.

One new development key had two configurations: r112 failed, then an adaptive r160 configuration recovered the key and enabled fresh signing.

Run-002 infrastructure and quarantine.

The first driver stopped at a Sage import before LLL/Babai, adding zero solver trials. A separate accidental cross-agent retry exited nonzero with no candidate and is quarantined as a non-result.

Untouched run-002.

One held-out key received one frozen S8 trial and no recovery: 0/160 selected intervals, 1/747 total intervals, and 214/747 public outputs fit. This negative is immutable.

Robust Family B development.

Six shared development keys received six trials and six recoveries, four from projected marginals and two from complete signatures. This selected S16.

Robust Family A development.

The same six keys received six separate trials and six recoveries; two needed BKZ-24. Family A ran after Family-B truth opened and is not blind evidence.

Robust environment stops and unused families.

Two Family-B launches stopped before solving because the NGCC environment was inactive; their reruns are already counted above. Families C and D were not run. The robust subtotal is six unique keys, twelve substantive A/B trials, twelve development successes, and two pre-solver environment stops.

Final campaign.

Run-003 used one untouched key, one fixed S16 trial, and one recovery with fresh signing. Run-004 was a zero-key, zero-trial terminal preseed infrastructure attempt. The initial run-005 package was VETOed preseed; its rev2 execution used one OS-random key, one fixed S16 trial, and one recovery with fresh signing. Thus the final campaign S16 count is 2/2, with no population-rate claim.

Public-package validation.

One earlier full harness generated a local key and signatures but stopped in controls before any solver attempt, so it contributes zero cryptanalytic trials. Five corrected full invocations then used five new local keys and five fixed trials; all five recovered the generated key and completed fresh signing without a failed cryptanalytic trial, solver retry, or cryptanalytic retuning. They shared the

fixed cryptanalytic configuration while noncryptanalytic harness and reporting code changed between validation snapshots. These add zero campaign trials or successes. The early-stop row is conservative because no failed result or time log survived. Quick, all-row, and determinism checks are zero-key public replays.

The demonstrated campaign query cost remains 16 signatures per attack execution. Development, campaign, infrastructure, and package-validation rows remain separate.

8 Scope and attribution

The attack is design-level: it follows from the intended opened-share data, abort condition, reconstruction-error bit, and formal modular verifier. It does not use a parser defect, malformed object, side channel, fault, secret label, or attack-specific acceptance rule. Only the Level-I $d = 32$ row is claimed, and the demonstrated procedure requires 16 accepted signatures.

Bittersweet itself identifies the carry-leakage surface. The lattice recovery uses established methods. ePrint 2026/250 by Baudrin, Heim Boissier, and Standaert is relevant generic power-of-two-LWR prior work, not an exact Bittersweet attack. We do not market modular-interval recovery, LLL, Babai, accepted-transcript leakage, order statistics, or the component methods as new.

9 Reproducibility

The required flat interface provides a quick public replay in `reproduce.py`, an independent replay of the sole affected row in `reproduce-all-rows.py`, and an optional full mode. The full mode generates a key locally, collects 16 complete signatures, verifies and projects them, recovers and freezes a candidate, compares only after the freeze, signs a distinct message, invokes the ordinary verifier, and runs negative controls. Dynamic results record OS, architecture, actual interpreter and argument vector, the logical command, dependency versions, runtime, peak RSS, local trials and recoveries, campaign-trial increment, and public replay counts. The quick public-only mode replays a sanitized interval instance and executes recovery rather than loading a stored key.

The run-005 campaign record supports AMD EPYC, Python 3.12.14, Singular 4.4.1, one CPU, and a 4 GiB cap. The sanitized public runner implements the needed inverse over $\mathbb{Z}/2^{96}\mathbb{Z}$ directly and does not use SageMath or Singular. Its hash lock targets CPython 3.12 on Linux x86-64 with glibc at least 2.17 and pins official binary wheels for fpyll 0.6.4 and cysignals 1.12.6; source builds are unverified. The locked install, imports, and an LLL smoke test passed in a fresh CPython 3.12.3 virtual environment. Measured reproducer runs used CPython 3.12.14. The exact-final-source full test completed one fresh-key execution in 184.47 seconds internal wall time at 83,336 KiB peak RSS on one worker, with all recovery and fresh-signing checks passing, 25/25 applicable query controls and 10/10 applicable fresh controls rejecting, and no raw candidate words emitted.

10 AI disclosure

Target discovery, history searching, scheme reconstruction, cryptanalysis, implementation, experiment execution, adversarial review, and writing were AI-assisted. Separate AI agents reviewed run-003. Another AI agent performed the one-trial fresh-context reproduction, and a further agent replayed its public-only result. A clean-context AI Publication Reviewer conditionally passed the frozen package; this AI review is not human verification. Independent human verification is pending. AI reproduction is not human verification.

References

- [1] B. Balon, G. Brian, S. Faust, C. Hazay, E. Micheli, and F.-X. Standaert, “Bittersweet Signatures: Bringing LWR to a Picnic for Hardware-Friendly MPC-in-the-Head,” IACR Cryptology ePrint Archive, Report 2026/397, version 1, 2026. Also in *Security and Cryptography for Networks (SCN 2026)*, LNCS 16810, pp. 96–120, 2026. <https://eprint.iacr.org/2026/397>; https://doi.org/10.1007/978-3-032-36264-3_6.

- [2] A. K. Lenstra, H. W. Lenstra, Jr., and L. Lovász, “Factoring polynomials with rational coefficients,” *Mathematische Annalen*, vol. 261, pp. 515–534, 1982.
- [3] L. Babai, “On Lovász’ lattice reduction and the nearest lattice point problem,” *Combinatorica*, vol. 6, pp. 1–13, 1986.
- [4] J. Baudrin, R. Heim Boissier, and F.-X. Standaert, “On the Concrete Hardness of LWR with a Power of Two Modulus,” IACR Cryptology ePrint Archive, Report 2026/250, 2026. <https://eprint.iacr.org/2026/250>.